Ex8 Report | Building a Web API

Nguyen Tieu Phuong 20210692

Source code

See the attached source code with this submission.

This web API is built using NodeJS and MongoDB.

The project is structured as follows:

```
/project-folder
  |-- /models
  |     |-- sensorData.js
  |     |-- user.js
  |-- /routes
  |     |-- sensorRoutes.js
  |     |-- userRoutes.js
  |-- app.js
  |-- swagger.js (optional)
```

Models

I have 2 models for the sensor data and for user. We specify them as how they are described in the problem statement.

sensorData.js

```
const mongoose = require('mongoose');

const sensorDataSchema = new mongoose.Schema({
    Id: { type: Number, required: true },
    SensorName: { type: String, required: true },
    SensorValue: { type: Number, required: true },
})

module.exports = mongoose.model('SensorData', sensorDataSchema);
```

user.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
    Id: { type: Number, required: true }, // Custom Id field
    UserName: { type: String, required: true },
    Password: { type: String, required: true },
    Email: { type: String, required: true },
});

module.exports = mongoose.model('User', userSchema);
```

Routes

We need to define the route (aka. Endpoints) for interacting with this API. Here I also include the Swagger documentation, but that's optional.

sensorRoutes.js

```
const express = require('express');
const router = express.Router();
const SensorData = require('../models/sensorData');

/**
 * @swagger
 * tags:
 *   name: SensorDatas
 *   description: API for managing SensorData
 */

/**
 * @swagger
 * /api/SensorDatas:
 *   get:
 *     summary: Get all SensorDatas
 *     tags: [SensorDatas]
 *     responses:
 *       200:
 *         description: A list of SensorDatas
 *         content:
 *           application/json:
 *             schema:
 *               type: array
 *               items:
 *                 $ref: '#/components/schemas/SensorData'
 */
```

```
router.get('/', async (req, res) => {
    try {
        const sensors = await SensorData.find();
        res.json(sensors);
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

/**
 * @swagger
 * /api/SensorDatas:
 *   post:
 *     summary: Create a new SensorData
 *     tags: [SensorDatas]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             $ref: '#/components/schemas/SensorData'
 *     responses:
 *       201:
 *         description: SensorData created successfully
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/SensorData'
 *       400:
 *         description: Bad Request
 */

router.post('/', async (req, res) => {
    const sensor = new SensorData({
        Id: req.body.Id, // Ensure unique ID
        SensorName: req.body.SensorName,
        SensorValue: req.body.SensorValue,
    });

    try {
        const newSensor = await sensor.save();
        res.status(201).json(newSensor);
    } catch (err) {
        res.status(400).json({ message: err.message });
    }
});
```

```
/**
 * @swagger
 * /api/SensorDatas/{id}:
 *   get:
 *     summary: Get a SensorData by ID
 *     tags: [SensorDatas]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *         description: The SensorData ID
 *     responses:
 *       200:
 *         description: SensorData retrieved successfully
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/SensorData'
 *       404:
 *         description: SensorData not found
 */
router.get('/:id', async (req, res) => {
    try {
        const sensor = await SensorData.findOne({ Id: req.params.id });
        if (!sensor) return res.status(404).json({ message: 'Sensor not
found' });
        res.json(sensor);
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

/**
 * @swagger
 * /api/SensorDatas/{id}:
 *   put:
 *     summary: Update a SensorData by ID
 *     tags: [SensorDatas]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *         description: The SensorData ID
 *     requestBody:
```

```
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             $ref: '#/components/schemas/SensorData'
 *     responses:
 *       200:
 *         description: SensorData updated successfully
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/SensorData'
 *       404:
 *         description: SensorData not found
 *       400:
 *         description: Bad Request
 */
router.put('/:id', async (req, res) => {
    try {
        const updatedSensor = await SensorData.findOneAndUpdate(
            { Id: req.params.id },
            req.body,
            { new: true }
        );
        if (!updatedSensor) return res.status(404).json({ message: 'Sensor
not found' });
        res.json(updatedSensor);
    } catch (err) {
        res.status(400).json({ message: err.message });
    }
});

/**
 * @swagger
 * /api/SensorDatas/{id}:
 *   delete:
 *     summary: Delete a SensorData by ID
 *     tags: [SensorDatas]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *         description: The SensorData ID
 *     responses:
 *       200:
 *         description: SensorData deleted successfully
```

```
 *       404:
 *         description: SensorData not found
 */
router.delete('/:id', async (req, res) => {
    try {
        const sensor = await SensorData.findOneAndDelete({ Id:
req.params.id });
        if (!sensor) return res.status(404).json({ message: 'Sensor not
found' });
        res.json({ message: 'Sensor deleted' });
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

module.exports = router;
```

userRoutes.js

```
const express = require('express');
const router = express.Router();
const User = require('../models/user');

/**
 * @swagger
 * tags:
 *   name: Users
 *   description: API for managing Users
 */

/**
 * @swagger
 * /api/Users:
 *   get:
 *     summary: Get all Users
 *     tags: [Users]
 *     responses:
 *       200:
 *         description: A list of Users
 *         content:
 *           application/json:
 *             schema:
 *               type: array
 *               items:
 *                 $ref: '#/components/schemas/User'
 */
```

```
router.get('/', async (req, res) => {
    try {
        const users = await User.find();
        res.json(users);
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

/**
 * @swagger
 * /api/Users:
 *   post:
 *     summary: Create a new User
 *     tags: [Users]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             $ref: '#/components/schemas/User'
 *     responses:
 *       201:
 *         description: User created successfully
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/User'
 *       400:
 *         description: Bad Request
 */
router.post('/', async (req, res) => {
    const user = new User({
        Id: req.body.Id, // Ensure unique ID
        UserName: req.body.UserName,
        Password: req.body.Password,
        Email: req.body.Email,
    });

    try {
        const newUser = await user.save();
        res.status(201).json(newUser);
    } catch (err) {
        res.status(400).json({ message: err.message });
    }
});

/**
```

```
 * @swagger
 * /api/Users/{id}:
 *   get:
 *     summary: Get a User by ID
 *     tags: [Users]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *         description: The User ID
 *     responses:
 *       200:
 *         description: User retrieved successfully
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/User'
 *       404:
 *         description: User not found
 */
router.get('/:id', async (req, res) => {
    try {
        const user = await User.findOne({ Id: req.params.id });
        if (!user) return res.status(404).json({ message: 'User not
found' });
        res.json(user);
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

/**
 * @swagger
 * /api/Users/{id}:
 *   put:
 *     summary: Update a User by ID
 *     tags: [Users]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *         description: The User ID
 *     requestBody:
 *       required: true
```

```
 *       content:
 *         application/json:
 *           schema:
 *             $ref: '#/components/schemas/User'
 *     responses:
 *       200:
 *         description: User updated successfully
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/User'
 *       404:
 *         description: User not found
 *       400:
 *         description: Bad Request
 */
router.put('/:id', async (req, res) => {
    try {
        const updatedUser = await User.findOneAndUpdate(
            { Id: req.params.id },
            req.body,
            { new: true }
        );
        if (!updatedUser) return res.status(404).json({ message: 'User not
found' });
        res.json(updatedUser);
    } catch (err) {
        res.status(400).json({ message: err.message });
    }
});

/**
 * @swagger
 * /api/Users/{id}:
 *   delete:
 *     summary: Delete a User by ID
 *     tags: [Users]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *         description: The User ID
 *     responses:
 *       200:
 *         description: User deleted successfully
 *       404:
```

```
 *         description: User not found
 */
router.delete('/:id', async (req, res) => {
    try {
        const user = await User.findOneAndDelete({ Id: req.params.id });
        if (!user) return res.status(404).json({ message: 'User not
found' });
        res.json({ message: 'User deleted' });
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

module.exports = router;
```

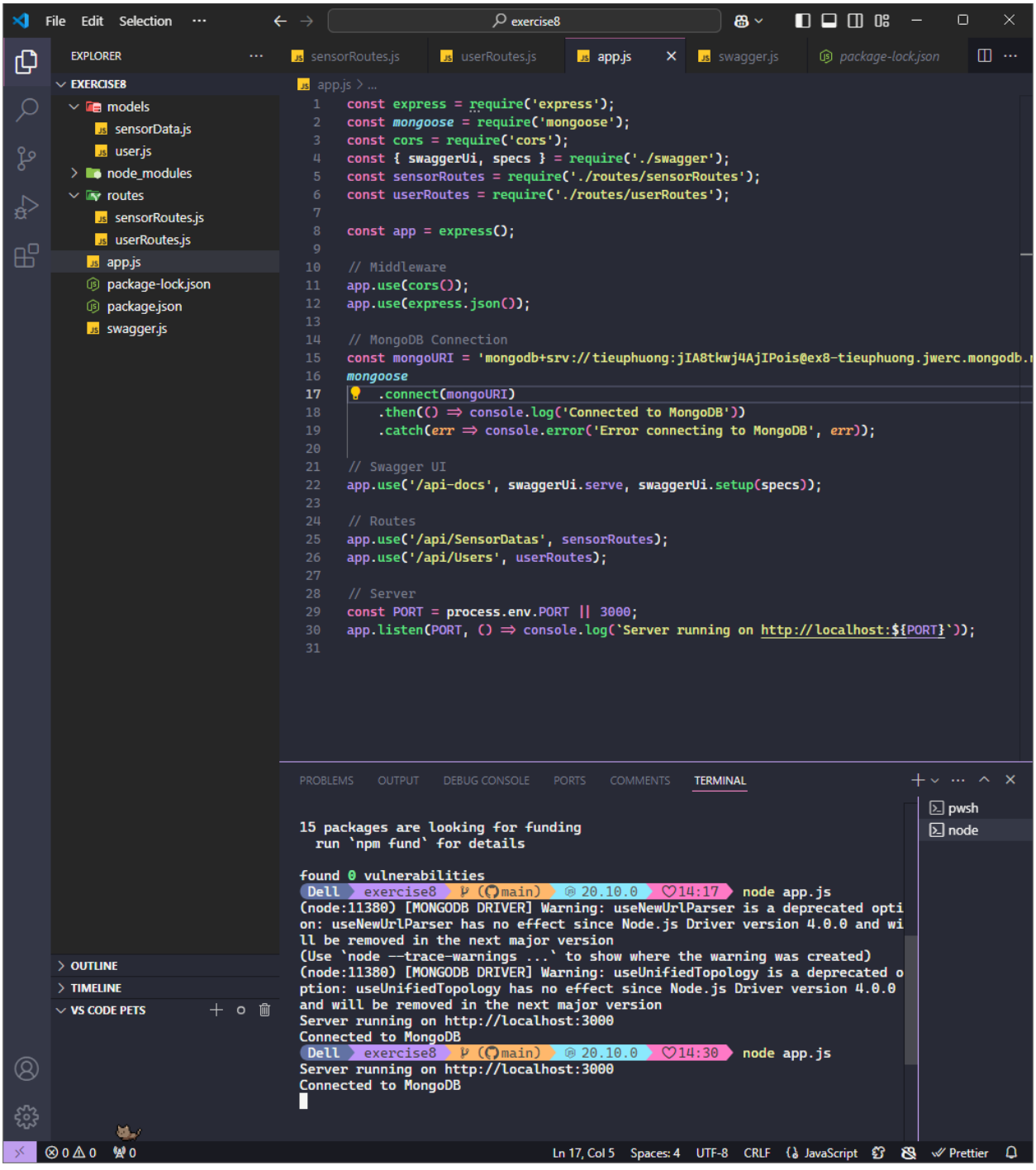
Main Application

App.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const { swaggerUi, specs } = require('./swagger');
const sensorRoutes = require('./routes/sensorRoutes');
const userRoutes = require('./routes/userRoutes');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// MongoDB Connection
const mongoURI = 'mongodb+srv://tieuphuong:jIA8tkwj4AjIPois@ex8-
tieuphuong.jwerc.mongodb.net/?retryWrites=true&w=majority&appName=Ex8-
TieuPhuong';
mongoose
    .connect(mongoURI)
    .then(() => console.log('Connected to MongoDB'))
    .catch(err => console.error('Error connecting to MongoDB', err));

// Swagger UI
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs));

// Routes
app.use('/api/SensorDatas', sensorRoutes);
```

```
app.use('/api/Users', userRoutes);

// Server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running on
http://localhost:${PORT}`));
```

Running the Web API

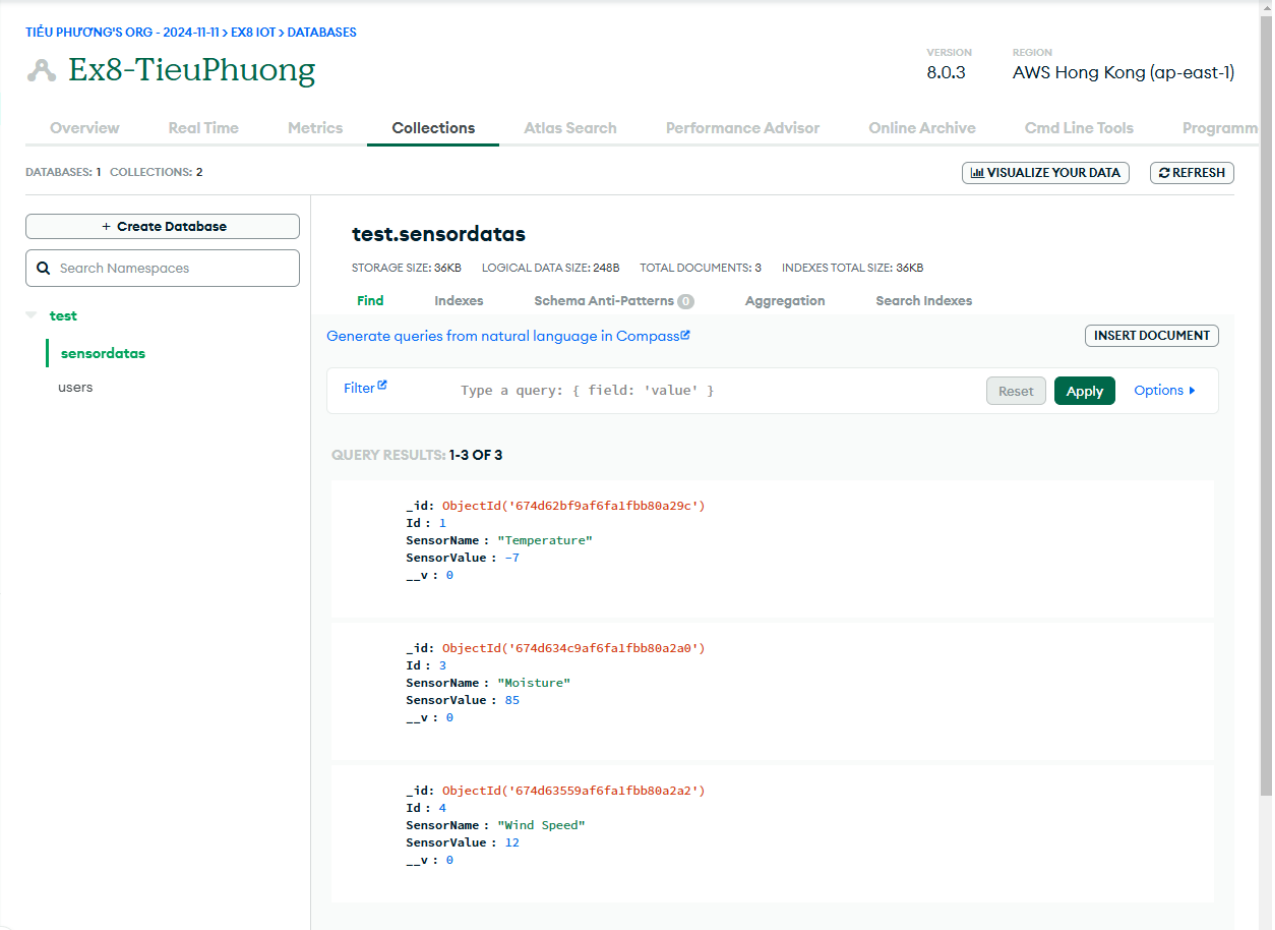


Server is running on port 3000.

Testing the API

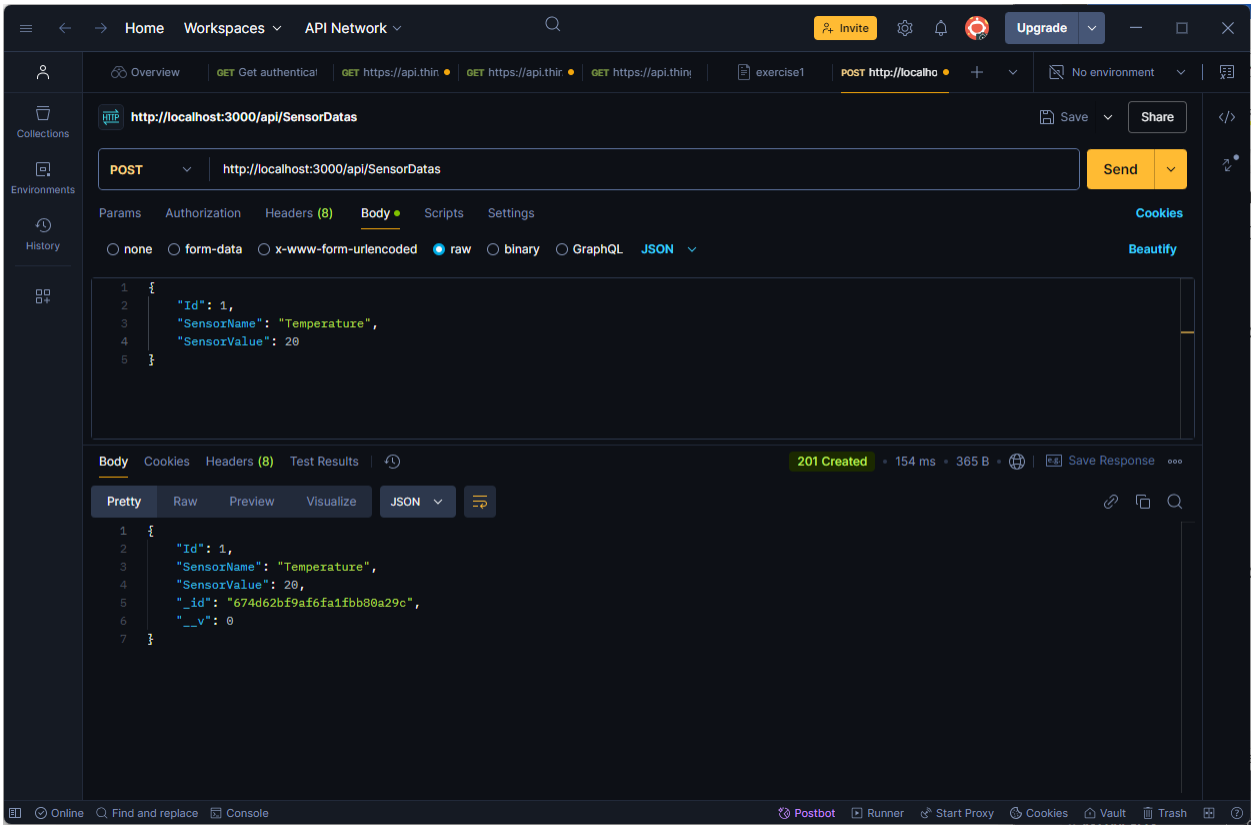
Note: MongoDB comes with a default `_id` field, auto generated string (not incremental). For this exercise, I create another, my own `Id` field that serves as the id for the users and sensor data.

SensorDatas

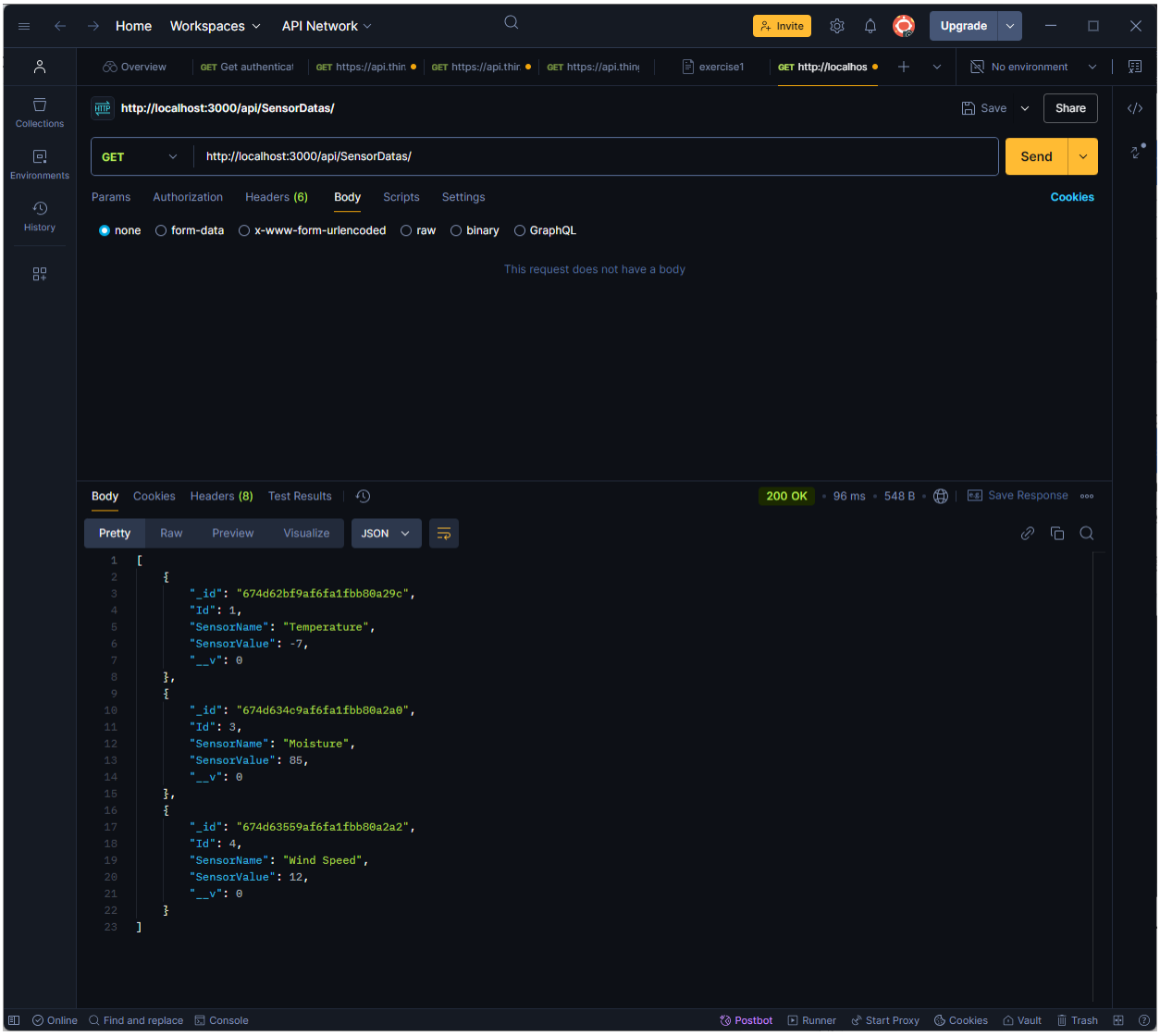


SensorDatas, as shown in MongoDB Atlas.

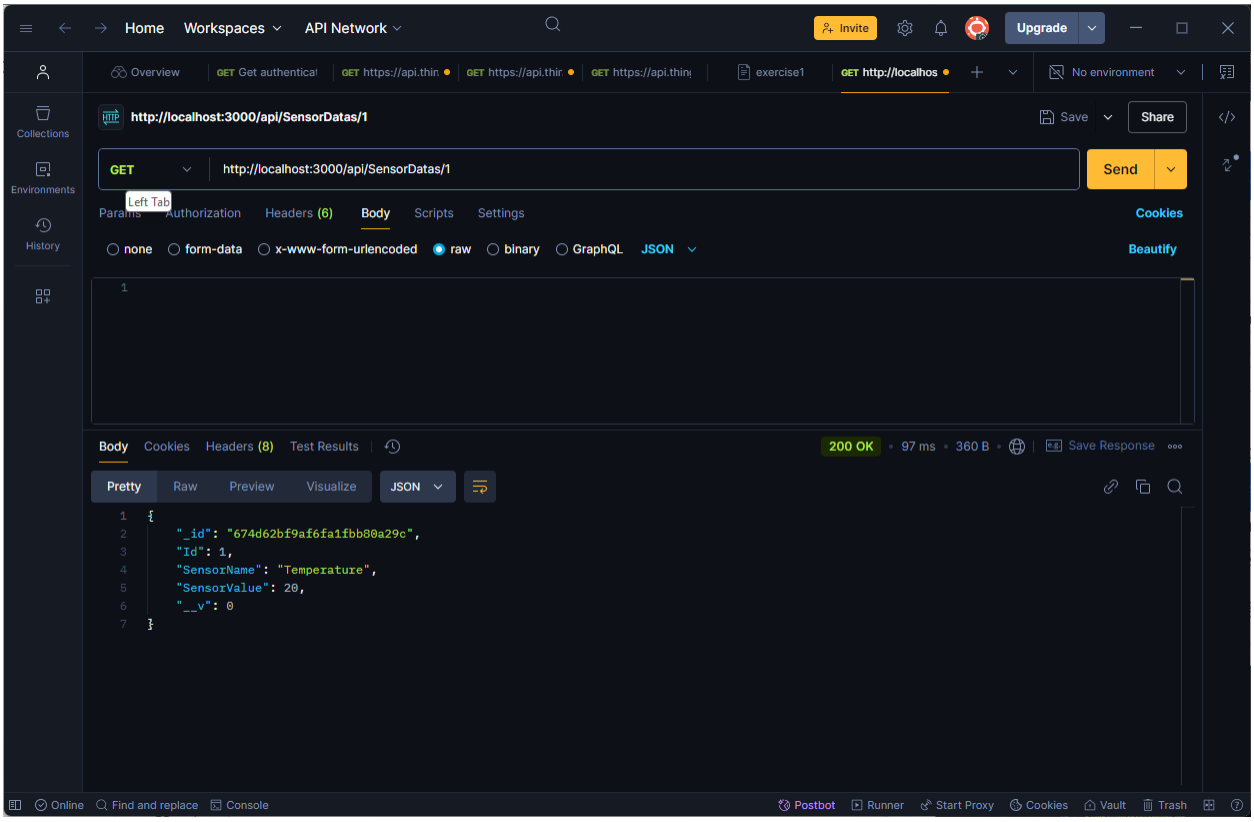
POST /api/SensorDatas



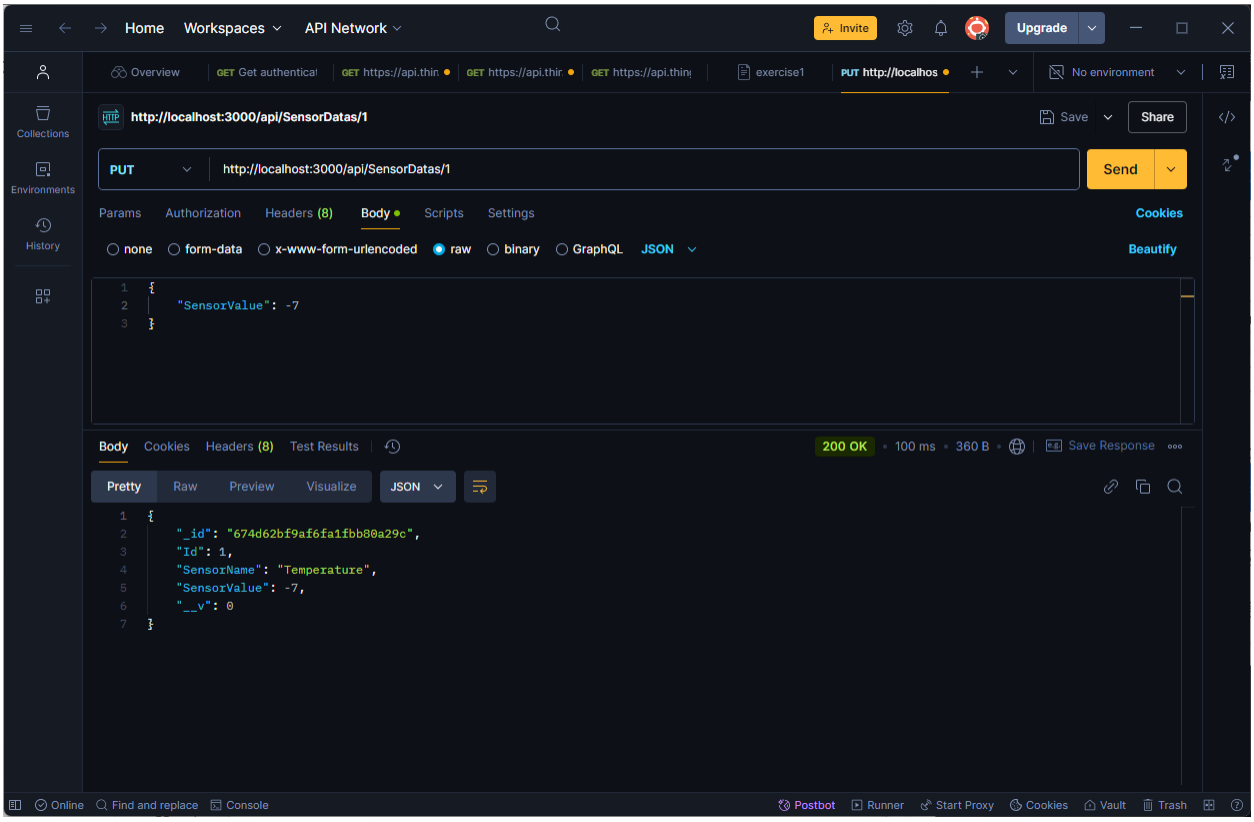
GET /api/SensorDatas



GET /api/SensorDatas/{id}

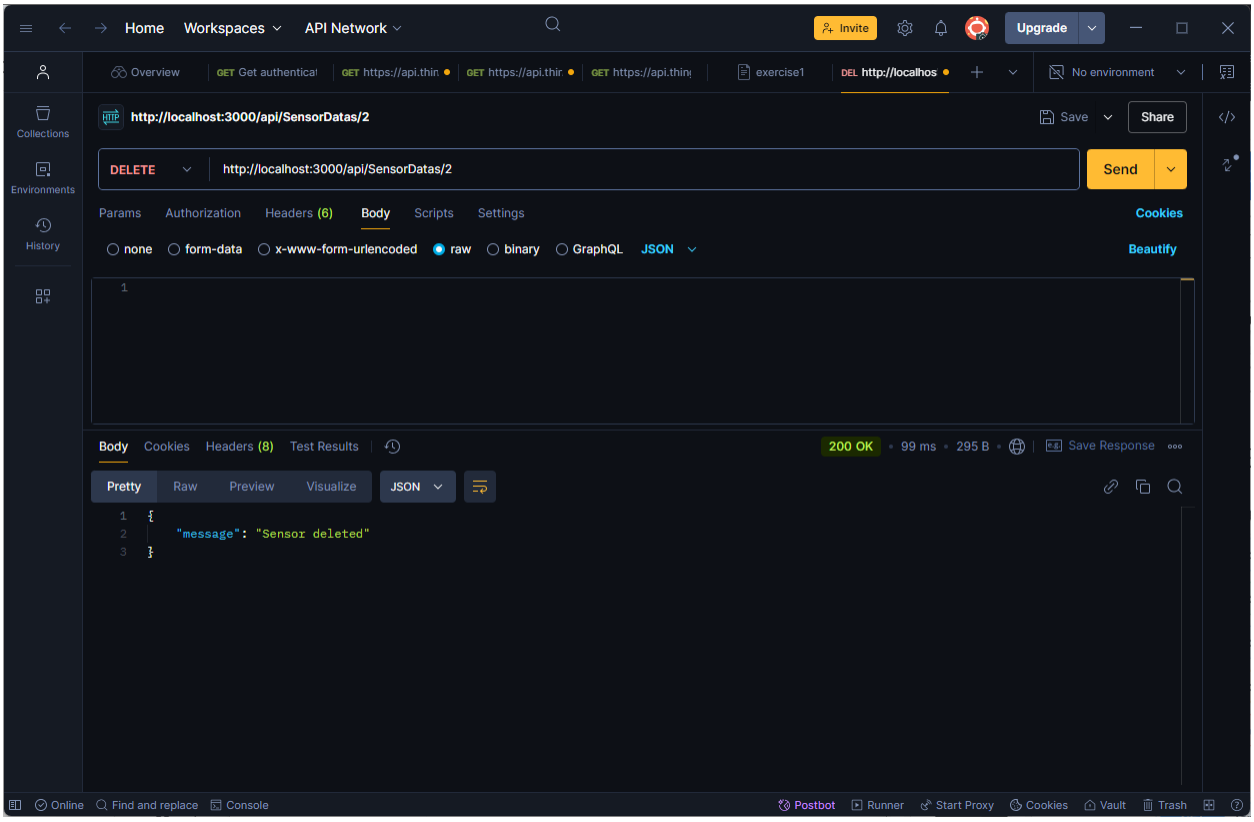


PUT /api/SensorDatas/{id}

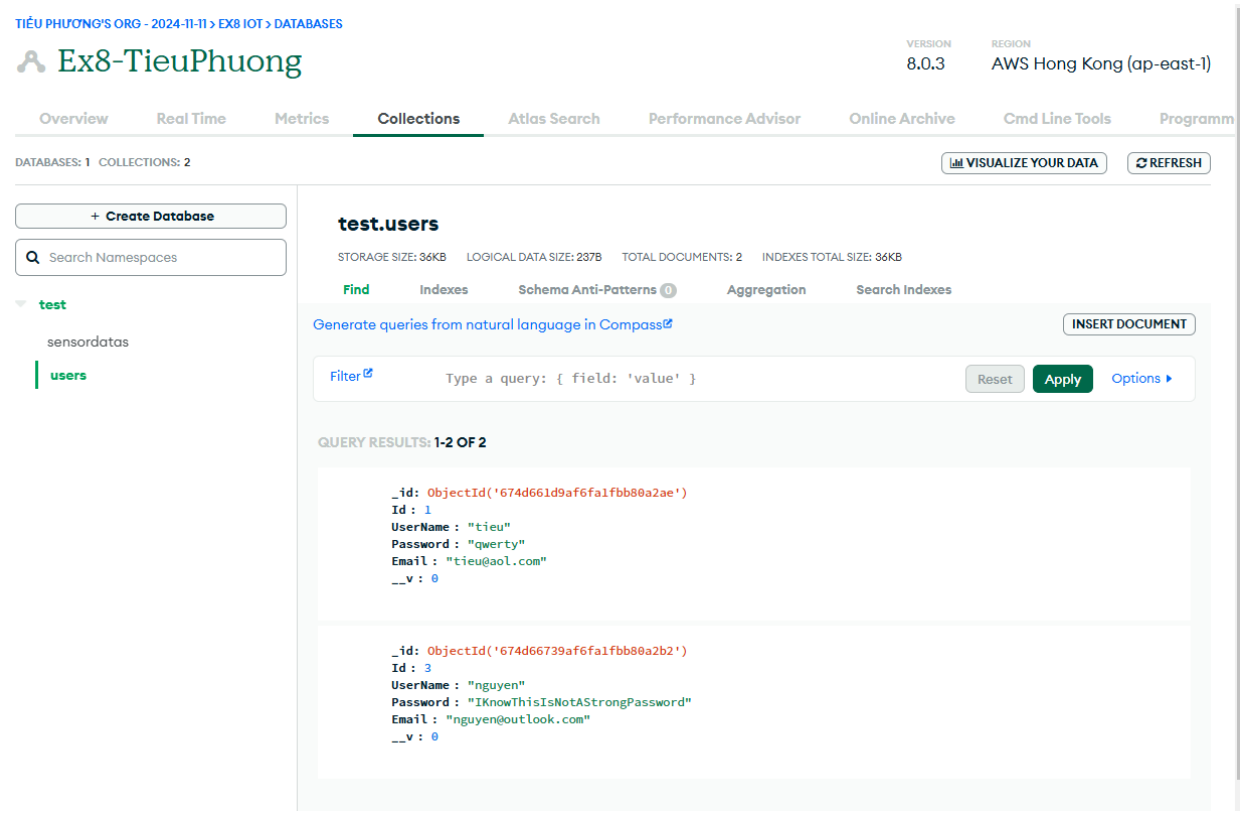


Change temperature value, for example.

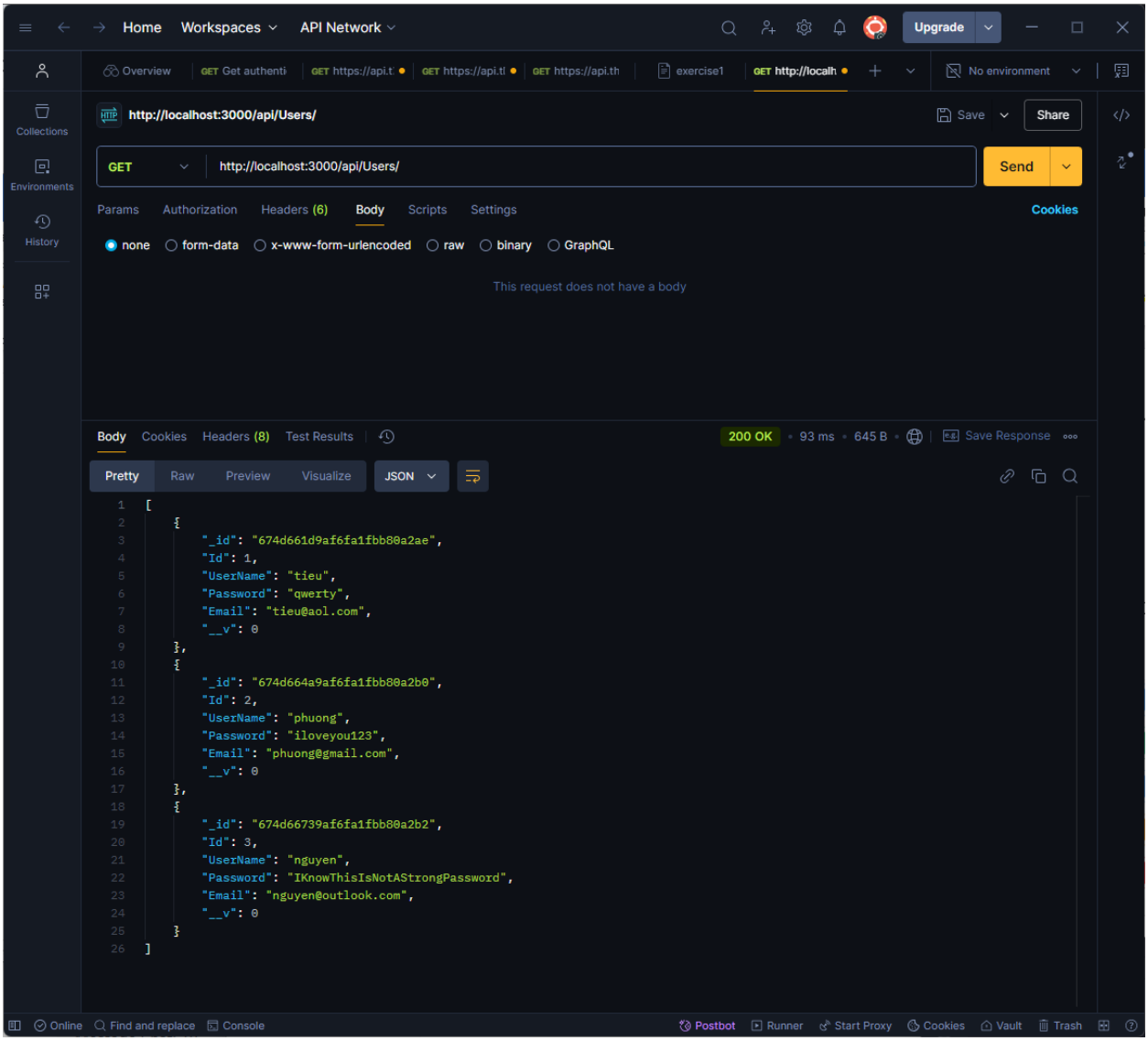
DELETE /api/SensorDatas/{id}



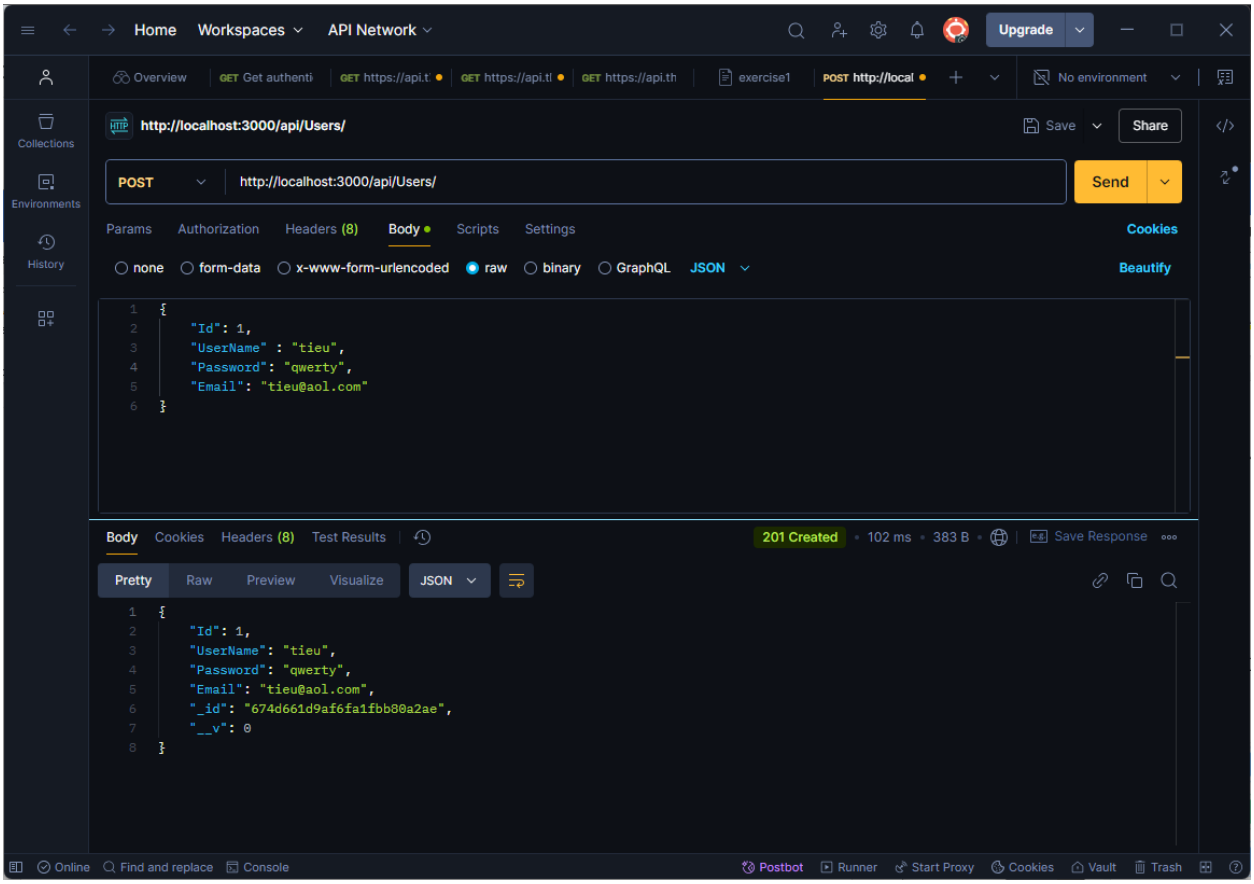
Users


TIÊU PHƯƠNG'S ORG - 2024-11-11 > EX8 IOT > DATABASES
Ex8-TieuPhuong
VERSION
8.0.3
REGION
AWS Hong Kong (ap-east-1)
Overview
Real Time
Metrics
Collections
Atlas Search
Performance Advisor
Online Archive
Cmd Line Tools
Programm
DATABASES: 1 COLLECTIONS: 2
VISUALIZE YOUR DATA
REFRESH
+ Create Database
Search Namespaces
test
sensordatas
users
test.users
STORAGE SIZE: 36KB
LOGICAL DATA SIZE: 237B
TOTAL DOCUMENTS: 2
INDEXES TOTAL SIZE: 36KB
Find
Indexes
Schema Anti-Patterns 0
Aggregation
Search Indexes
Generate queries from natural language in Compass
INSERT DOCUMENT
Filter
Type a query: { field: 'value' }
Reset
Apply
Options
QUERY RESULTS: 1-2 OF 2
_id: ObjectId('674d661d9af6fa1fbb80a2ae')
Id : 1
UserName : "tieu"
Password : "qwerty"
Email : "tieu@aol.com"
__v : 0
_id: ObjectId('674d66739af6fa1fbb80a2b2')
Id : 3
UserName : "nguyen"
Password : "IKnowThisIsNotAStrongPassword"
Email : "nguyen@outlook.com"
__v : 0

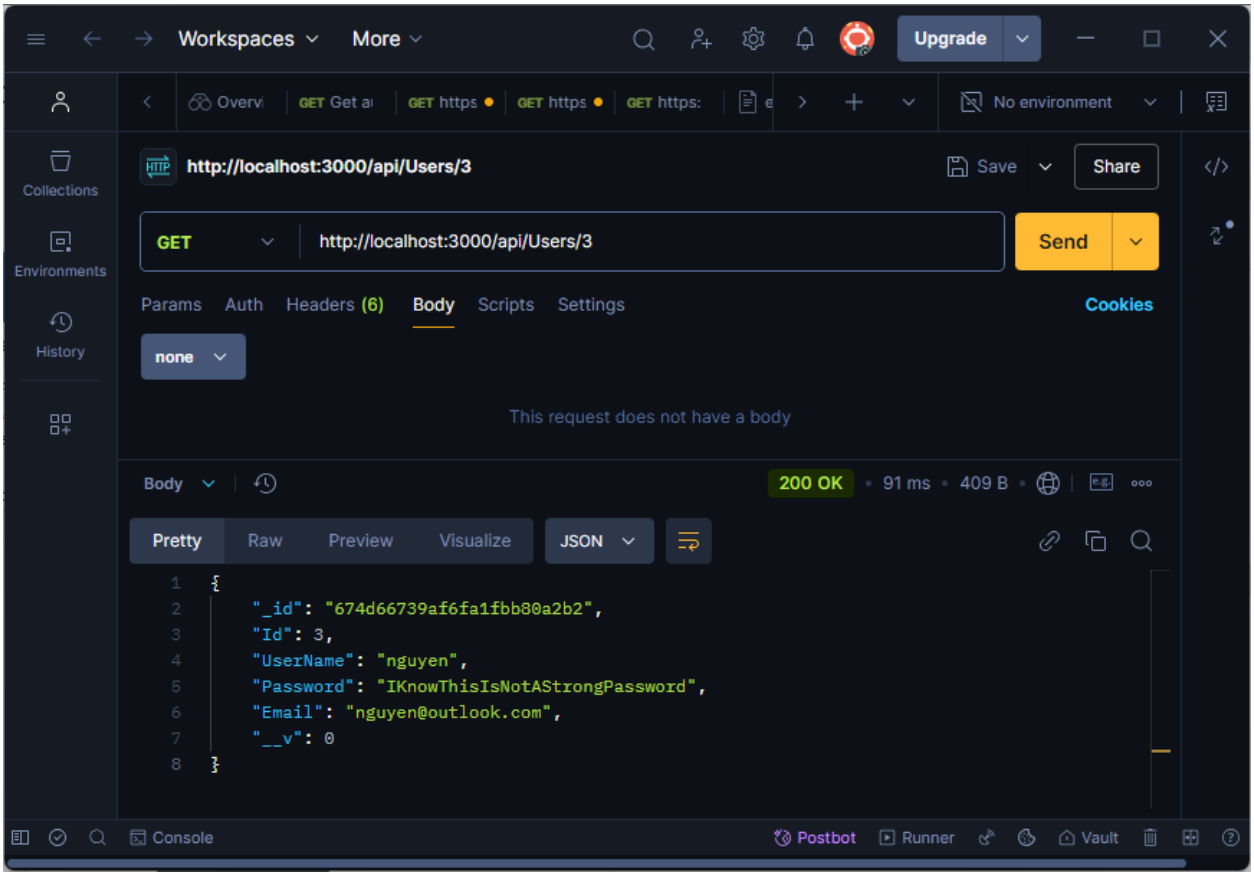
GET /api/Users



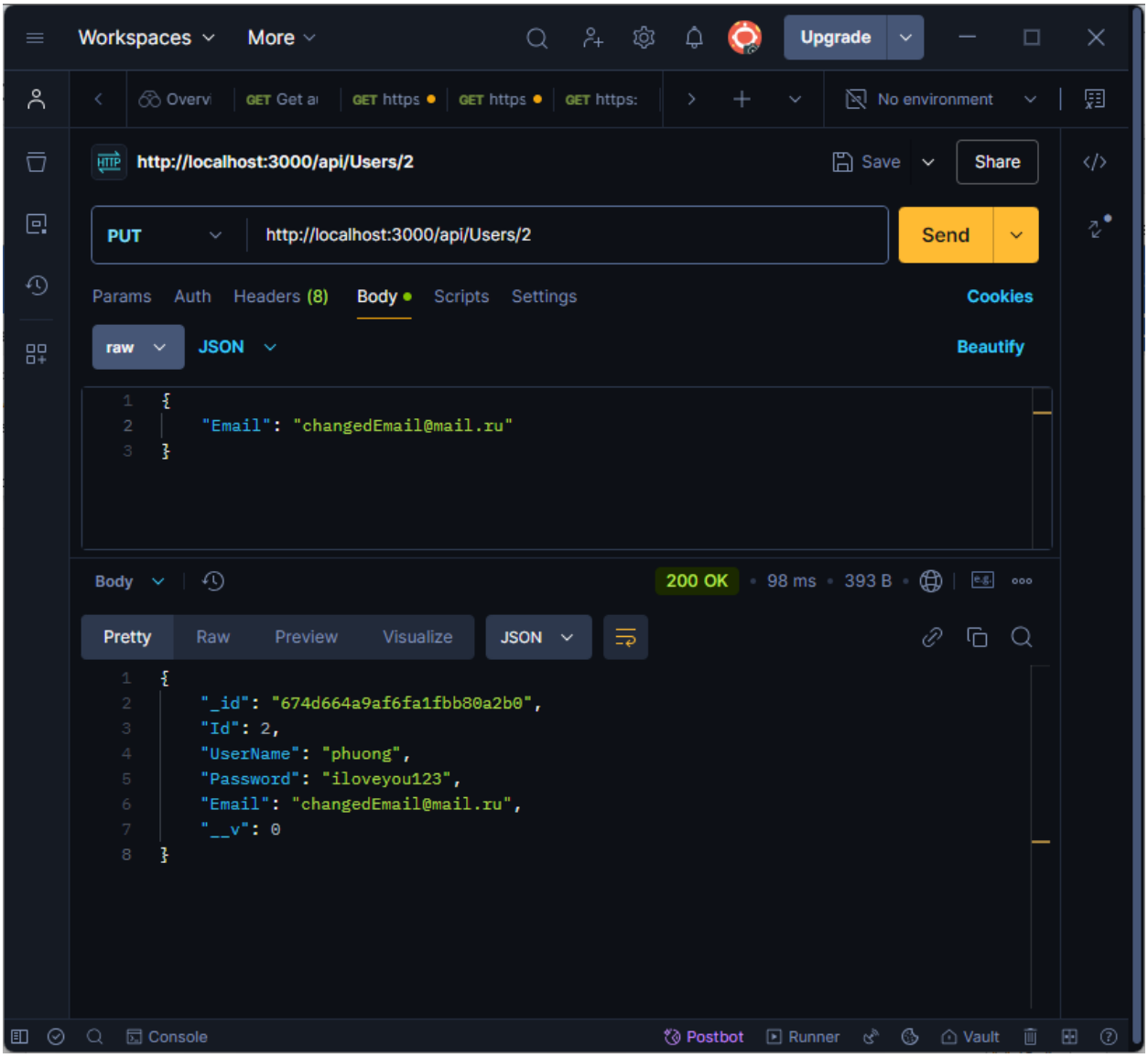
POST /api/Users



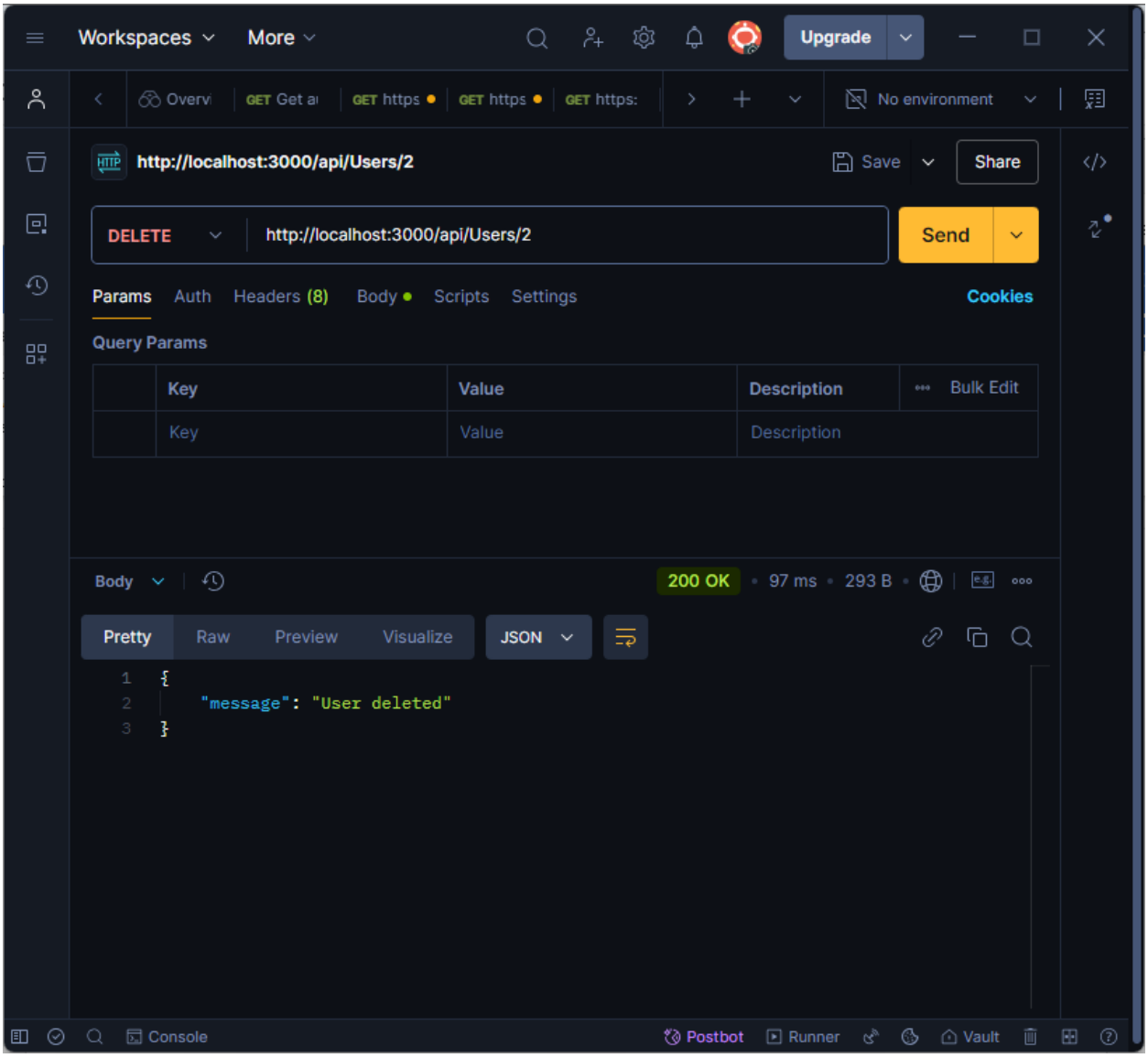
GET /api/Users/{id}



PUT /api/Users/{id}



DELETE /api/Users/{id}



THE END